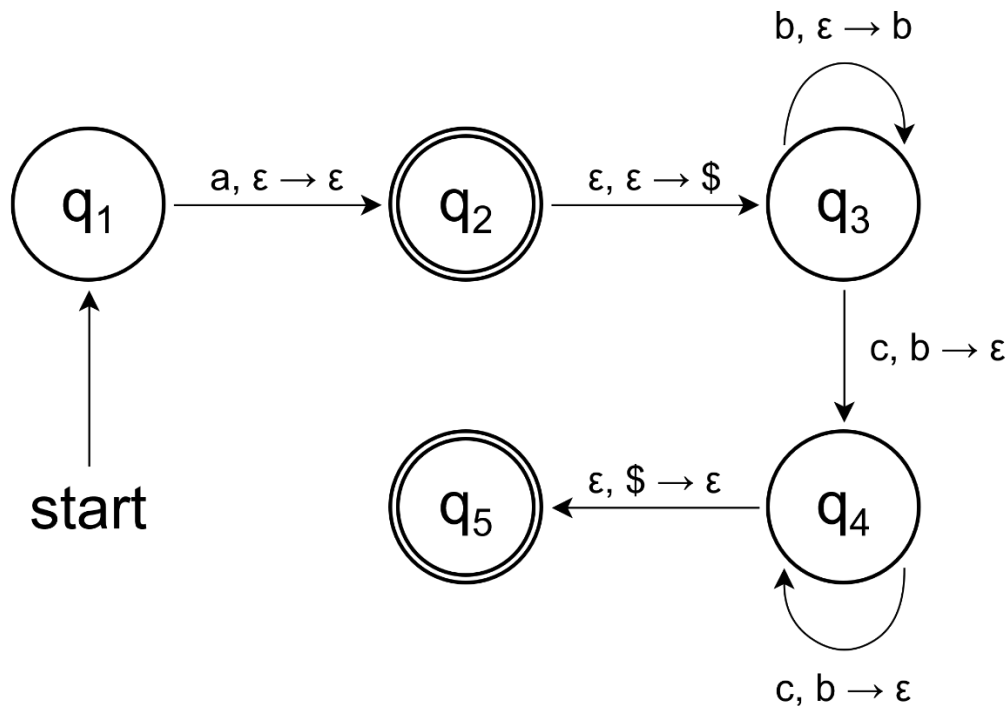


Problem 1: Context-free languages

Problem 1a

Context-free grammar $G_1 = (V, \Sigma, R, S)$ where $V = \{S\}$ are the placeholders, $\Sigma = \{a, b, c\}$ is the output domain, the start symbol is S where $S \in V$, and $R = \{S \rightarrow Sbc \mid a\}$ is a set of rules. This grammar G_1 generates the language $L_1 = \{ab^n c^n \mid n \geq 0\}$, i.e. $L(G_1) = L_1$.

Problem 1b



Problem 1c

The simplest way to show that $L_2 = \{a^i b^j c^k \mid i, j, k \geq 0, \text{ and } i + j = k\}$ is context-free is by either creating a PDA that recognizes this language, or by creating a CFG that generates this language. I will go with the CFG route which I define as $G_2 = (V, \Sigma, R, S)$ where $V = \{S, T\}$, $\Sigma = \{a, b, c\}$, $R = \{S \rightarrow aSc \mid T \mid \varepsilon, T \rightarrow bTc \mid \varepsilon\}$. As you can see G_2 creates the language L_2 .

Problem 2: Pumping lemmas

Problem 2a

First, we assume that the language is regular. Thus, there exists a DFA with p states (also called pumping length) that accepts all the inputs of that language. According to the pumping lemma all these inputs can be divided into 3 parts, $w = xyz$, and there are 3

conditions that always hold. Now, all we have to do is come up with an input that is accepted by the definition of L_2 , but breaks one of the pumping lemma's conditions, i.e. a counter example. I chose $w = a^p c^p$ which according to the 3rd condition " $|xy| \leq p$ " means that xy in w consists of only and all as . In addition to that, due to the 2nd condition " $|y| > 0$ " y must consist of only as , i.e. $y \neq \varepsilon$, but there is no such rule for x . Due to this, the 1st condition " $xy^i z \in L_2$ for all $i \geq 0$ " breaks when $i \geq 2$ because it is then that there isn't any valid decomposition of $w = xyz$ such that the amount of cs is the sum of the amount of as and bs (as required by the definition of $L_2 = \{a^i b^j c^k \mid i, j, k \geq 0, \text{ and } i + j = k\}$) since as are increasing in count whilst that of cs isn't.

Problem 2b

To prove that the language $L_3 = \{a^i b^j c^k \mid i, j, k \geq 0 \text{ and } i \times j = k\}$ is not context-free, for the sake of contradiction, we assume that it is context-free. This means that there exists a PDA with p transition states that accepts the language. Here we also use the pumping lemma (which is different to that of the regular languages) according to which we can split the input into 5 parts, $w = uvxyz$. This time I chose input $w = a^p b^p c^{p^2}$ where due to the 3rd condition " $|vxy| \leq p$ " and 2nd condition " $|vy| > 0$ " we get 5 ways to split it up, i.e. 5 cases:

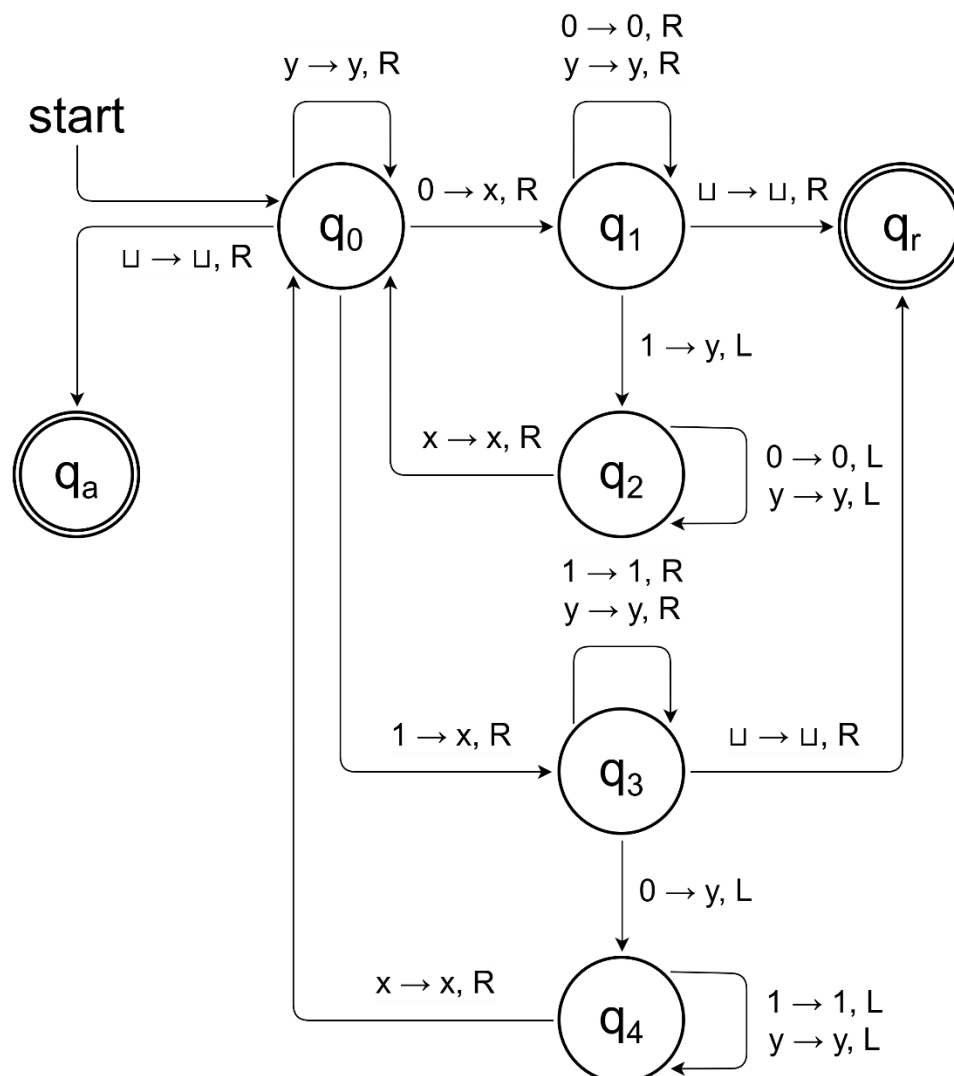
1. u is an ε , vxy contains only and all as where, in terms of the count of as , x is at least an ε and at least either v or y contains an a , and z contains only and all of both bs and cs .
2. u contains only and all as , vxy contains only and all bs where, in terms of the count of bs , x is at least an ε and at least either v or y contains a b , and z contains only and all cs .
3. u contains only and all of both as and bs , vxy contains only but not all cs where, in terms of the count of cs , x is at least an ε and at least either v or y contains a c , whereas z contains the rest of cs .
4. u contains only but not all as , vxy contains both as and bs , but not all of either, where in terms of the count of both as and bs , x is at least an ε and at least either v or y contains an a or a b , and z contains the rest of bs and all cs .
5. vxy contains both bs and cs , but not all of either, where in terms of the count of both bs and cs , x is at least an ε and at least either v or y contains a b or a c , u contains all of as and the rest of bs , and z contains only and the rest of cs .

In all the cases the 1st condition " $uv^i xy^i z \in L_3$ for all $i \geq 0$ " breaks when $i \geq 2$ because it is then that there isn't any valid decomposition of $w = uvxyz$ such that amount of cs is the product of as and bs (as required by the definition of L_3). In cases 1, 2, and 4 the amount of cs remains constant whilst the amount of as and/or bs doesn't. In case 3 the amount of

both as and bs remains constant whilst the amount of cs doesn't. Lastly, in case 5 both the amount of bs and cs grow at a constant rate whilst the amount of as doesn't which is wrong. That is because the amount of cs must grow at an increasing rate in accordance with the growth of the amount of bs , which it does not. More formally, pumping increases both the number of bs and cs . Now, let t and s be the amount of as and bs added by the increase. Then, the string can be described as having p amount of as , $p + t$ amount of bs , and $p^2 + s$ amount of cs . Due to the product requirement, the amount of as and bs with respect to cs must be and can be described as $p \times (p + t) = p^2 + p \times t = p^2 + s$. This implies that $p \times t = s$, but since $pt \geq p \geq s$ this does not hold in the general case, i.e. only in a restricted case when $pt = s$.

Problem 3: Turing machines

Problem 3a



Problem 3b

First, I set up an environment where the machine can also sweep right to left where a blank cell acts as an indicator to stop sweeping, switch sweeping side, or both. Simultaneously, we accept the base case where an input is empty.

1. Read, then write to the first cell a blank.
2. If in 1st step the cell contained a blank, accept.
3. If in 1st step the cell contained a 0, sweep right and replace first blank cell with a 0.
4. If in 1st step the cell contained a 1, sweep right and replace first blank cell with a 1.
5. Sweep left until the first blank cell is reached.

Next, for each 1 that is found and crossed off we find and cross off two 0s. If at any point during this process the amount of either is found to be insufficient, we reject the input.

6. Sweep right crossing off the first 1 that is found.
7. If in 6th step we reach an empty cell before a 1 was found, reject.
8. Sweep left until the first blank cell is reached.
9. Sweep right crossing off two 0s.
10. If in 9th step we reach a blank first, reject.

Lastly, we verify that everything is crossed off, if it is we accept, if it isn't we go to 5th step which sets up the next cross off (steps 6 to 10) and this verification (steps 11 – 13).

11. Sweep left until a 0 or a 1 is found.
12. If in 11th step we reach a blank first, accept.
13. Go to 5th step.